Introduzione a OpenJML

Elvinia Riccobene - Valentina Ciriani

Annotazioni JML

•Le annotazioni JML vengono aggiunte come commenti nel file .java tra

/*@ . . . @*/ , o dopo //@

attenzione agli spazi: /* @ non viene riconosciuto!

• Le condizioni sono scritte come espressioni booleane di java con alcuni operatori in più:

\result, \old(), !, &&, ||, ==>, <==>, <=!=>, \forall, \exists, ...

e alcune parole chiave

requires, ensures, invariant, ...

Notazione base di JML

JML permette di aggiungere come commenti particolari ad un classe Java:

- precondizioni ai metodi
  - //@ requires <espressione booleana>;
- postcondizioni ai metodi
  - //@ ensures <espressione booleana>;
- invarianti di classe
  - //@ public invariant <espressione booleana>;

Notazione base di JML: Esempio

```
public class Contatore {
  /*@ public invariant counter >= 0; @*/
  private int counter;

  Contatore() { counter = 0; }

  //@ requires counter >= a;
  //@ ensures counter == \old(counter) - a;
  //@ ensures \result == b;
  public int decrementa(int a) {
  int b; counter-= a; b=counter; return b; }
…}
```

Deve essere vero:
- dopo ogni costruttore
- prima di ogni metodo
- dopo ogni metodo

Quali sono gli errori?

Notazione base di JML: Esempio

```
public class Contatore {
   /*@ public invariant counter >= 0; @*/
   private int counter;

   Contatore() { counter = 0; }

   //@ requires counter >= a;
   //@ ensures counter == \old(counter) - a;
   //@ ensures \result == b;
   public int decrementa(int a) {
   int b; counter-= a; b=counter; return b; }
…}
```

- counter è privato: non lo posso usare!
- b è dichiarato dentro un metodo: non lo posso usare!

Notazione base di JML: Esempio corretto

```
public class Contatore {
   /*@ public invariant counter >= 0; @*/
   private /*@ spec_public @*/ int counter;

   Contatore() { counter = 0; }

   //@ requires counter >= a;
   //@ ensures counter == \old(counter) - a;
   //@ ensures \result == counter;
   public int decrementa(int a) {
   int b; counter-= a; b=counter; return b; }
…}
```

Posso usare:
- counter
- a
- \result
- \old(counter)

Non posso usare:
- b

Espressioni booleane proposizionali

JML permette di usare molti connettivi booleani proposizionali: !, &&, ||, ==>, <==>, <=!=>

- ! è il NOT: !A è vero se e solo se A è falso
- && è l'AND: (A && B) è vero se e solo se A e B sono entrambi veri
- || è l'OR: (A || B) è vero se e solo se almeno uno tra A e B è vero
- ==> è l'implicazione: (A ==> B) è *falso* se e solo se A è vero e B è falso
  NB (A==>B) è equivalente a (!A || B)

Espressioni booleane proposizionali

- <==> è l'equivalenza: (A <==> B) è vero se e solo se A e B hanno lo stesso valore booleano (A e B sono entrambi veri, oppure A e B sono entrambi falsi)
- <=!=> è lo XOR: (A <=!=> B) è vero se e solo se A e B hanno valori booleani diversi (A è vero e B è falso, oppure A è falso e B è vero)

De Morgan

Attenzione al NOT quando è combinato con AND e OR

Leggi di De Morgan:

!(P && Q) = !P || !Q

!(P || Q) = !P && !Q

Esempio: *entrambi* gli elementi in posizione i e j dell'array A non sono strettamente positivi:

!(A[i] > 0) && !(A[j] > 0) oppure !(A[i] > 0 || A[j] > 0)

NB !(A[i] > 0 && A[j] > 0) è sbagliato! Perché?

Modellazione di formule condizionali

Supponiamo di voler modellare in JML una postcondizione della funzione

```
static int minimo(int a, int b)
```

- se a <= b allora minimo restituisce a
- altrimenti (ovvero a>b): minimo restituisce b

Come lo modello?

```
/*@ ensures ((a <= b) ==> (\result == a))
  @        && ((a > b) ==> (\result == b));
  @*/
static int minimo(int a, int b){
if (a <= b) return a; else return b;}
```

Modellazione di formule condizionali

```
/*@ ensures ((a <= b) ==> (\result == a))
  @         && ((a > b) ==> (\result == b));
  @*/
```

è equivalente a:

```
/*@ ensures ((a <= b) && (\result == a))
  @           || ((a > b) && (\result == b));
  @*/
```

Modellazione di formule condizionali

Se ho più di due possibili output?

Esempio: una funzione

```
static int confronta(int a, int b)
```

che restituisce:

- -1 se a < b,
- 1 se a >b e
- 0 se a=b

Come modello la postcondizione?

```
/*@ ensures ((a < b) ==> (\result == -1))
  @         && ((a > b) ==> (\result ==  1))
  @         && ((a == b) ==> (\result == 0))
  @*/
static int confronta(int a, int b){
    if (a < b) return -1;
    else if (a > b) return 1;
    else return 0;}
```

Quantificatori

Nelle espressioni di JML posso inserire anche quantificatori:

- universali \forall - per ogni elemento
- esistenziali \exists – esiste un elemento
- generali \sum, \product, \min, \max
- numerici \num_of

La sintassi è

(<quant.> <tipo> <var>; <range predicate>; <espressione>)

Esempio:

Tutti gli elementi dell'array A sono strettamente positivi

∀i ((i>= 0 && i< A.length)) ==> A[i]>0)

```
(\forall int i; i>= 0 && i< A.length; A[i]>0)
```

Quantificatori \forall e \exists

restituiscono un booleano

Esempi:

Tutti gli elementi dell'array A sono uguali a 0

```
(\forall int i; i>= 0 && i< A.length; A[i]==0)
```

E' un valore booleano

Almeno un elemento dell'array A è uguale a 0

```
(\exists int i; i>= 0 && i< A.length; A[i]==0)
```

Nessun elemento dell'array A è uguale a 0:

```
!(\exists int i; i>= 0 && i< A.length; A[i]==0)
```

oppure?

~~`!(\forall int i; i>= 0 && i< A.length; A[i]==0)`~~

oppure?

```
(\forall int i; i>= 0 && i< A.length; A[i]!=0)
```

Quantificatore \num_of

restituisce un intero quindi va usato con == oppure >, < ecc…

Esempio : il numero di elementi maggiori di 0 in A è esattamente 5

```
(\num_of int i; i>= 0 && i< A.length; A[i]>0)== 5
```

E' un valore booleano

Esattamente un elemento dell'array A è uguale a 0

~~(\exists int i; i>= 0 && i< A.length; A[i]==0)~~

oppure?

```
(\num_of int i; i>= 0 && i< A.length; A[i]==0)==1
```

Quantificatori \sum \product \min \max

restituiscono un valore numerico quindi vanno usati con == oppure >, < ecc…

Esempio:

```
(\sum int i; i>= 0 && i< A.length; A[i])== 5
```

E' l'elemento da sommare!

24 è il minore di tutti elementi maggiori di 0 dell'array A:

```
(\min int i; i>= 0 && i< A.length && A[i] > 0; A[i])== 24
```

il prodotto degli elementi in posizione pari dell'array A è maggiore di 34

```
(\product int i; i>= 0 && i< A.length && i%2==0; A[i])> 34
```

Quantificatori con matrici

Dopo il quantificatore possiamo mettere più variabili

Esempio:
si consideri una matrice bidimensionale `int[][] M;`
con `M = new int[4][2];`
Il più piccolo elemento della matrice M è 3

```
(\min int i, j;
i >=0 && i < 4 && j>=0 && j < 2;
M[i][j]) == 3
```

Quantificatori con matrici

Esempio:
si consideri una matrice bidimensionale `boolean[][] M;`
con `M = new boolean[5][6];`

Tutti gli elementi della matrice M tranne M[0][0] sono falsi

```
(\forall int i, j;
i >=0 && i < 5 && j >=0 && j < 6 && !(i==0 && j==0);
!M[i][j])

(!i==0 && !j==0)
```

Quantificatori annidati

E' possibile usare un quantificatore come espressione di un altro.

Esempio. la tabella 3x3 deve contenere tutti i numeri da 1 a 9

```
public class Tabella9 {
private /*@spec_public@*/  int[][] tabella;
// per ogni intero tra 1 e 9, c'è una casella che lo contiene
/*@ ensures (\forall int k; k >= 1 && k <= 9;
@  (\exists int i, j; i >= 0 && i <= 2 && j >= 0 &&
@       j <= 2; tabella[i][j] == k) );
@*/
public Tabella9(){ //costruttore
    tabella = new int [3][3];
    int k=1;
    for (int i=0; i<3, i++)
        for (int j=0; j<3, j++)
            tabella[i][j] = k++;}
}
```

1	2	3
4	5	6
7	8	9

Quantificatori annidati

E' corretto scrivere?:

```
public class Tabella9 {
private /*@spec_public@*/  int[][] tabella;
// per ogni intero tra 1 e 9, c'è una casella che lo contiene
/*@ public invariant (\exists int k; k >= 1 && k <= 9;
@        (\forall int i, j; i >= 0 && i <= 2 && j >= 0 && j <= 2;
@        tabella[i][j] == k));
@*/
```

Non va bene perché stiamo dicendo che tutti gli elementi della tabella hanno lo stesso valore k (tra 1 e 9) ad esempio:

3	3	3
3	3	3
3	3	3

Quantificatori annidati

E' corretto scrivere?:

```
public class Tabella9 {
private /*@spec_public@*/  int[][] tabella;

// per ogni intero tra 1 e 9, c'è una casella che lo contiene
/*@ public invariant (\forall int i, j; i >= 0 && i <= 2 &&
@       j >= 0 && j <= 2; (\exists int k; k >= 1 && k <= 9;
@       tabella[i][j] == k));
@*/
```

Non va bene perché dice che ogni elemento deve contenete un intero da 1 a k. Ovvero anche questa tabella rispetta l'invariante:

1	2	1
1	1	3
1	2	1

Tool utilizzato in laboratorio

Per JML in laboratorio adottiamo il plugin OpenJML:
http://jmlspecs.sourceforge.net/

L'indirizzo dell'update site per installarlo da eclipse è:
http://jmlspecs.sourceforge.net/openjml-updatesite

Attenzione: Il plugin non funziona con Java 8. E' necessario avere Java 7 installato ed eseguire eclipse usando Java 7

Tool utilizzato in laboratorio

Per avere tutto già predisposto in una versione di eclipse stand-alone per windows (senza bisogno di altre istallazioni), scaricare:

http://fmse.di.unimi.it/sw/eclipsePSS_win.zip

Scompattare il file zip ed eseguire eclipse_java7.bat (non eclipse.exe!)

L'installazione contiene anche JUnit ed Eclemma che useremo per la parte di testing

Tool utilizzato in laboratorio

- Per abilitare le specifiche JML: selezionare "Enable JML" dal menu contestuale del progetto (tasto destro sul progetto)

- Per controllare che la sintassi sia corretta: selezionare il bottone con la tazza gialla

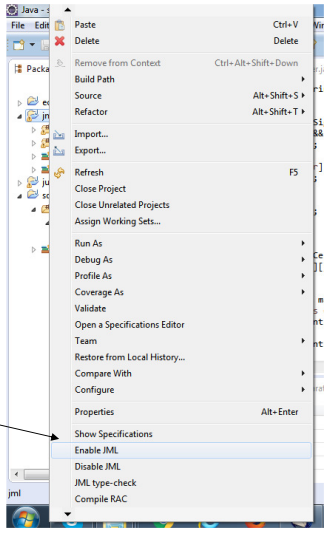
- Per compilare il codice con JML: selezionare il bottone "RAC". E' necessario ricompilare il codice dopo ogni modifica delle specifiche (cioè il codice non viene ricompilato in automatico con JML). La corretta sequenza per compilare ed eseguire una specifica JML è:

1. salvataggio del file
2. esecuzione di RAC
3. esecuzione del codice come "Java Application"

Come abilitare JML

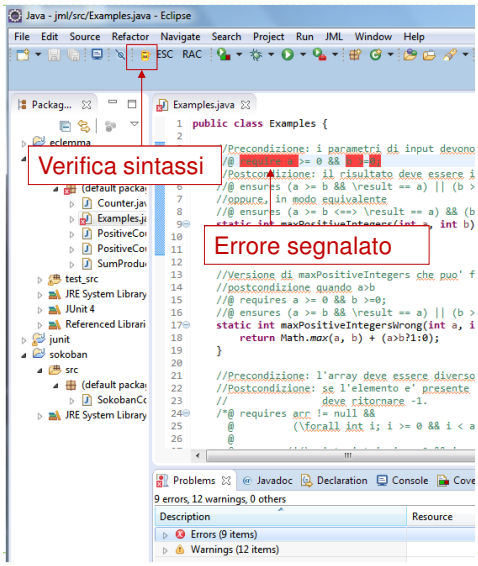
Da effettuare la prima volta che si usa JML sul nuovo progetto (tasto dx)

Enable JML

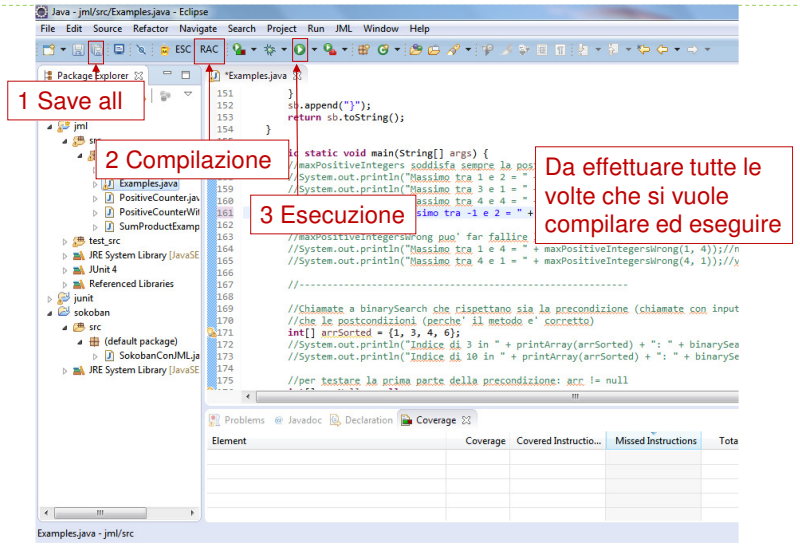


Verifica sintassi

Da effettuare quando si vuole verificare la sintassi JML



Compilazione ed esecuzione



Metodi puri

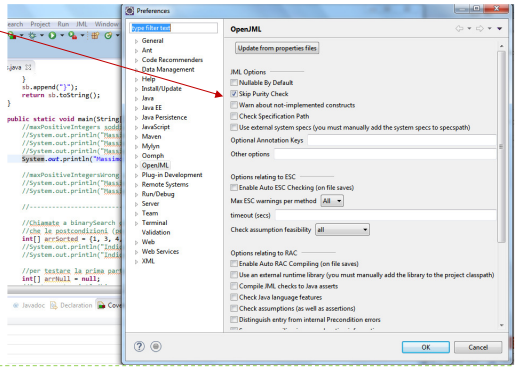
- Un metodo è puro se non modifica lo stato (ovvero gli attributi)
- Per identificare un metodo come puro lo si può dichiarare con il modificatore
  `/*@ pure @*/`
- Solo i metodi puri possono essere utilizzati nelle precondizioni e nelle postcondizioni

```
public class Punto2D {
   protected double x;
   protected double y;
   public /*@ pure @*/ double getX() {
        return x;
   }
   //@ ensures getX() == \old(getX()) + a;
   public void incrX(double a) {
        x += a;
   }
}
```

Togliere il purity check

Come suggerito anche dagli sviluppatori di OpenJML, conviene deselezionare il controllo sulla purezza dei metodi (a causa di troppi falsi positivi)

Selezionare la checkbox: Window/Preferences/OpenJML/Skip Purity Check



Quantificatori con più variabili e OpenJML

- OpenJML ha problemi a gestire quantificatori con più variabili
- Soluzione: basta annidarli

- Esempio. OpenJML da errore con:

```
/*@ ensures (\forall int i, j; i >= 0 && i <= 2 && j >= 0 &&
@        j <= 2; tabella[i][j] == 3);
@*/
```

Basta trasformarlo in:

```
/*@ ensures (\forall int i; i >= 0 && i <= 2; (\forall int j;
@        j >= 0 && j <= 2; tabella[i][j] == 3));
@*/
```

Quantificatori con più variabili e OpenJML

- OpenJML ha problemi a gestire quantificatori con più variabili
- Soluzione: basta annidarli

- Esempio. OpenJML da errore con:

```
/*@ ensures (\forall int i, j; i >= 0 && i <= 2 && j >= 0 &&
@        j <= 2; tabella[i][j] == 3);
@*/
```

Basta trasformarlo in:

```
/*@ ensures (\forall int i; i >= 0 && i <= 2; (\forall int j;
@        j >= 0 && j <= 2; tabella[i][j] == 3));
@*/
```

Quantificatori con più variabili e OpenJML

- Attenzione nel range del quantificatore annidato non si possono mettere vincoli alla variabile del quantificatore esterno
- Esempio:

```
/*@ ensures (\forall int i; i >=0 && i < 5 ;(\forall int j; j >=0
@            && j < 5 && !(i == 1 && j == 1); !board[i][j]));
@*/
```

Vincolo su i nel range di j

deve essere trasformato in:

```
/*@ ensures (\forall int i; i >=0 && i < 5 ;(\forall int j; j >=0
@            && j < 5; !(i == 1 && j == 1) ==> !board[i][j]));
@*/
```

Metto il vincolo su i nell'espressione di j con un ==>

Quantificatori con più variabili e OpenJML

- Posso però usare *i* nel range di *j* per vincolare *j* (non posso vincolare *i* nel range di *j*)
- Esempio:

```
/*@ ensures (\forall int i; i >=0 && i < arr.length;
@   (\forall int j; j >=0 && j < arr[i].length; arr[i][j] > 0))
@*/
```

Vincolo su j che utilizza i nel range

va bene

Quantificatori con più variabili e OpenJML

- **\num_of** è un po' diverso dagli altri:

- Esempio. OpenJML da errore con:

```
/*@ ensures (\num_of int i, j; i >= 0 && i <= 2 && j >= 0 &&
@       j <= 2; tabella[i][j] == 0);
@*/
```

Basta trasformarlo in:

```
/*@ ensures (\sum int i; i >= 0 && i <= 2; (\num_of int j;
@       j >= 0 && j <= 2; tabella[i][j] == 0));
@*/
```

Perché calcolo `\num_of` su tutte le righe `i` della tabella e poi, per avere il risultato complessivo, mi basta sommare (con un `\sum`) tutti i risultati dei `\num_of`

Esempi ed esercizio

Esempi iniziali con metodi statici:

Examples.java

SumProductExamples.java

Esempio semplice di classe con attributo e metodo puro:

Counter.java

Esempio di classe con invariante:

PositiveCounterWithInvariant.java

Esercizio di esame

Sokoban partendo da Sokoban.java sul sito (esercizioSokobanJML.pdf)

Esercizi per casa

Risolvere (annotazioni JML) gli esercizi di esame:

- Gioco dell'8 (esercizioGiocoOttoJML.pdf)
- Knight's tour (esercizioKnightTourJML.pdf)